

2.3 Distributed Systems

There are two major trends in recent developments in hardware technology:

- Computer systems with multiple CPUs are entering even small offices, notably multiprocessing systems running operating systems such as IBM OS/2 Warp, Microsoft Windows NT, or UNIX.
- Local area networks connecting hundreds of heterogeneous computers have become commonplace.

Nowadays, even small companies are using distributed systems. But what are the advantages of distributed systems that make them so interesting? Tanenbaum [Tan92] suggests the following:

Economics. Computer networks that incorporate both PCs and workstations offer a better price/performance ratio than mainframe computers.

Performance and Scalability. According to the Sun Microsystems philosophy 'The network is the computer', distributed applications are capable of using resources available on a network. A huge increase in performance can be gained by using the combined computing power of several network nodes. In addition—at least in theory—multiprocessors and networks are easily scalable.

Inherent distribution. Some applications are inherently distributed, for example database applications that follow a Client-Server model.

Reliability. In most cases, a machine on a network or a CPU in a multiprocessor system can crash without affecting the rest of the system. Central nodes such as file servers are notable exceptions to this, but can be protected by backup systems.

Distributed systems, however, have a significant drawback [Tan92]: 'Distributed systems need radically different software than do centralized systems'. This is the major technical reason why consortia such as the Object Management Group (OMG) and companies such as Microsoft have developed their own technologies for distributed computing.

We introduce three patterns related to distributed systems in this category:

- The *Pipes and Filters* pattern (53) provides a structure for systems that process a stream of data. Each processing step is encapsulated in a filter component. Data is passed through pipes between adjacent filters. Recombining filters allows you to build families of related systems.

This pattern is more often used for structuring the functional core of an application than for distribution, so we describe it in a different category—see Section 2.2, *From Mud to Structure*.

- The *Microkernel* pattern (171) applies to software systems that must be able to adapt to changing system requirements. It separates a minimal functional core from extended functionality and customer-specific parts. The microkernel also serves as a socket for plugging in these extensions and coordinating their collaboration.

Microkernel systems employ a Client-Server architecture in which clients and servers run on top of the microkernel component. The main benefit of such systems, however, is in design for adaptation and change. We therefore place the pattern description in another category—see Section 2.5, *Adaptable Systems*.

Platforms such as Microsoft OLE (Object Linking and Embedding) [Bro94] and OMG's CORBA (Common Object Request Broker Architecture) [OMG92] share a common software architecture, from which we have abstracted the Broker pattern:

- The *Broker* pattern (99) can be used to structure distributed software systems with decoupled components that interact by remote service invocations. A broker component is responsible for coordinating communication, such as forwarding requests, as well as for transmitting results and exceptions.

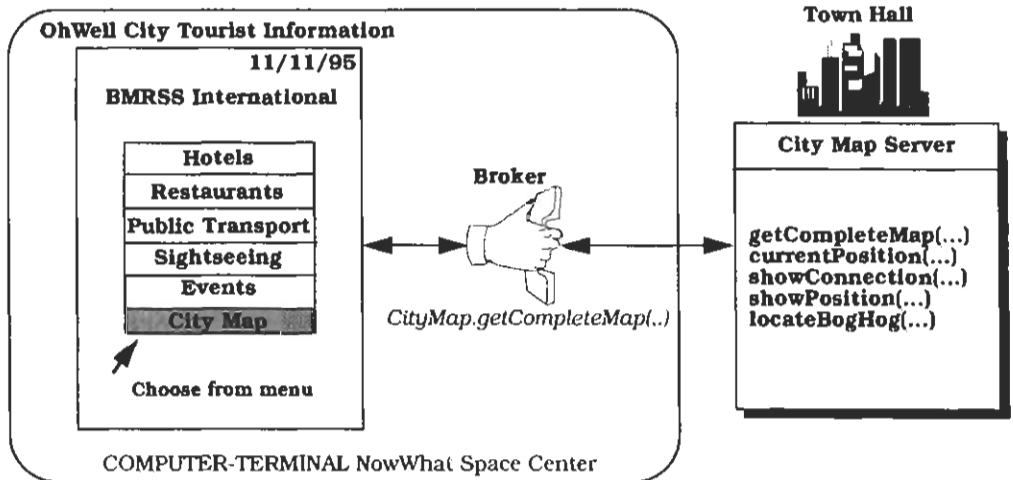
There are three groups of developers who can benefit by using the Broker pattern:

- Those working with an existing Broker system who are interested in understanding the architecture of such systems.
- Those who want to build 'lean' versions of a Broker system, without all the bells and whistles of a full-blown OLE or CORBA.
- Those who plan to implement a fully-fledged Broker system, and therefore need an in-depth description of the Broker architecture.

Broker

The *Broker* architectural pattern can be used to structure distributed software systems with decoupled components that interact by remote service invocations. A broker component is responsible for coordinating communication, such as forwarding requests, as well as for transmitting results and exceptions.

Example Suppose we are developing a city information system (CIS) designed to run on a wide area network. Some computers in the network host one or more services that maintain information about events, restaurants, hotels, historical monuments or public transportation. Computer terminals are connected to the network. Tourists throughout the city can retrieve information in which they are interested from the terminals using a World Wide Web (WWW) browser. This front-end software supports the on-line retrieval of information from the appropriate servers and its display on the screen. The data is distributed across the network, and is not all maintained in the terminals.



We expect the system to change and grow continuously, so the individual services should be decoupled from each other. In addition, the terminal software should be able to access services without having to know their location. This allows us to move, replicate, or

migrate services. One solution is to install a separate network that connects all terminals and servers, leading to an *intranet* system. Such an approach, however, has several disadvantages: not every information provider wants to connect to a closed intranet, and even more importantly, available services should also be accessible from all over the world. We therefore decide to use the Internet as a better means of implementing the CIS system.

Context Your environment is a distributed and possibly heterogeneous system with independent cooperating components.

Problem Building a complex software system as a set of decoupled and inter-operating components, rather than as a monolithic application, results in greater flexibility, maintainability and changeability. By partitioning functionality into independent components the system becomes potentially distributable and scalable.

However, when distributed components communicate with each other, some means of inter-process communication is required. If components handle communication themselves, the resulting system faces several dependencies and limitations. For example, the system becomes dependent on the communication mechanism used, clients need to know the location of servers, and in many cases the solution is limited to only one programming language.

Services for adding, removing, exchanging, activating and locating components are also needed. Applications that use these services should not depend on system-specific details to guarantee portability and interoperability, even within a heterogeneous network.

From a developer's viewpoint, there should essentially be no difference between developing software for centralized systems and developing for distributed ones. An application that uses an object should only see the interface offered by the object. It should not need to know anything about the implementation details of an object, or about its physical location.

Use the Broker architecture to balance the following *forces*:

- Components should be able to access services provided by others through remote, location-transparent service invocations.
- You need to exchange, add, or remove components at run-time.

- The architecture should hide system- and implementation-specific details from the users of components and services.

Solution Introduce a *broker* component to achieve better decoupling of clients and servers. Servers register themselves with the broker, and make their services available to clients through method interfaces. Clients access the functionality of servers by sending requests via the broker. A broker's tasks include locating the appropriate server, forwarding the request to the server and transmitting results and exceptions back to the client.

By using the Broker pattern, an application can access distributed services simply by sending message calls to the appropriate object, instead of focusing on low-level inter-process communication. In addition, the Broker architecture is flexible, in that it allows dynamic change, addition, deletion, and relocation of objects.

The Broker pattern reduces the complexity involved in developing distributed applications, because it makes distribution transparent to the developer. It achieves this goal by introducing an object model in which distributed services are encapsulated within objects. Broker systems therefore offer a path to the integration of two core technologies: distribution and object technology. They also extend object models from single applications to distributed applications consisting of decoupled components that can run on heterogeneous machines and that can be written in different programming languages.

Structure The Broker architectural pattern comprises six types of participating components: *clients*, *servers*, *brokers*, *bridges*, *client-side proxies* and *server-side proxies*.

A *server*¹⁰ implements objects that expose their functionality through interfaces that consist of operations and attributes. These interfaces are made available either through an interface definition language (IDL) or through a binary standard. The Implementation section contains a comparison of these approaches. Interfaces typically

10. In this pattern description *servers* are responsible for implementing services. In an object-oriented approach every service is realized by one or more *objects*. We use the term *server object* to emphasize the fact that such a server appears to other components as an object in the object-oriented sense.

group semantically-related functionality. There are two kinds of servers:

- Servers offering common services to many application domains.
- Servers implementing specific functionality for a single application domain or task.

➡ The servers in our CIS example comprise WWW servers that provide access to HTML (Hypertext Markup Language) pages. WWW servers are implemented as `httpd` daemon processes (hypertext transfer protocol daemon) that wait on specific ports for incoming requests. When a request arrives at the server, the requested document and any additional data is sent to the client using data streams. The HTML pages contain documents as well as CGI (Common Gateway interface) scripts for remotely-executed operations on the network host—the remote machine from which the client received the HTML-page. A CGI script may be used to allow the user fill out a form and submit a query, for example a search request for vacant hotel rooms. To display animations on the client's WWW browser, Java 'applets' are integrated into the HTML documents. For example, one of these Java applets animates the route between one place and another on a city map. Java applets run on top of a virtual machine that is part of the WWW browser. CGI scripts and Java applets differ from each other: CGI scripts are executed on the server machine, whereas Java applets are transferred to the WWW browser and then executed on the client machine. □

Clients are applications that access the services of at least one server. To call remote services, clients forward requests to the broker. After an operation has executed they receive responses or exceptions from the broker.

The interaction between clients and servers is based on a dynamic model, which means that servers may also act as clients. This dynamic interaction model differs from the traditional notion of Client-Server computing in that the roles of clients and servers are not statically defined. From the viewpoint of an implementation, you can consider clients as applications and servers as libraries—though other implementations are possible. Note that clients do not need to know the location of the servers they access. This is important,

because it allows the addition of new services and the movement of existing services to other locations, even while the system is running.

➡ In the context of the Broker pattern, the clients are the available WWW browsers. They are not directly connected to the network. Instead, they rely on Internet providers that offer gateways to the Internet, such as Compuserve. WWW browsers connect to these workstations, using either a modem or a leased line. When connected they are able to retrieve data streams from httpd servers, interpret this data and initiate actions such as the display of documents on the screen or the execution of Java applets. □

Class Client	Collaborators • Client-side Proxy • Broker	Class Server	Collaborators • Server-side Proxy • Broker
Responsibility • Implements user functionality. • Sends requests to servers through a client-side proxy.		Responsibility • Implements services. • Registers itself with the local broker. • Sends responses and exceptions back to the client through a server-side proxy.	

A *broker* is a messenger that is responsible for the transmission of requests from clients to servers, as well as the transmission of responses and exceptions back to the client. A broker must have some means of locating the receiver of a request based on its unique system identifier. A broker offers APIs (Application Programming Interfaces) to clients and servers that include operations for registering servers and for invoking server methods.

When a request arrives for a server that is maintained by the local broker¹¹, the broker passes the request directly to the server. If the server is currently inactive, the broker activates it. All responses and exceptions from a service execution are forwarded by the broker to the client that sent the request. If the specified server is hosted by another broker, the local broker finds a route to the remote broker

11. In this pattern description we distinguish between *local* and *remote* brokers. A local broker is running on the machine currently under consideration. A remote broker is running on a remote network node.

and forwards the request using this route. There is therefore a need for brokers to interoperate.

Depending on the requirements of the whole system, additional services—such as *name services*¹² or *marshaling support*¹³—may be integrated into the broker.

Class	Collaborators
Broker	• Client • Server • Client-side Proxy • Server-side Proxy • Bridge
Responsibility • (Un-)registers servers. • Offers APIs. • Transfers messages. • Error recovery. • Interoperates with other brokers through bridges. • Locates servers.	

➤ A broker in our CIS example is the combination of an Internet gateway and the Internet infrastructure itself. Every information exchange between a client and a server must pass through the broker. A client specifies the information it wants using unique identifiers called URLs (Universal Resource Locators). By using these identifiers the broker is able to locate the required services and to route the requests to the appropriate server machines. When a new server machine is added, it must be registered with the broker. Clients and servers use the gateway of their Internet provider as an interface to the broker. □

Client-side proxies represent a layer between clients and the broker. This additional layer provides transparency, in that a remote object appears to the client as a local one. In detail, the proxies allow the hiding of implementation details from the clients such as:

12. Name services provide associations between names and objects. To resolve a name, a name service determines which server is associated with a given name. In the context of Broker systems, names are only meaningful relative to a *name space*.

13. Marshaling is the semantic-invariant conversion of data into a machine-independent format such as ASN.1 (Abstract Syntax Notation) or ONC XDR (eXternal Data Representation). Unmarshaling performs the reverse transformation.

- The inter-process communication mechanism used for message transfers between clients and brokers.
- The creation and deletion of memory blocks.
- The marshaling of parameters and results.

In many cases, client-side proxies translate the object model specified as part of the Broker architectural pattern to the object model of the programming language used to implement the client.

Server-side proxies are generally analogous to Client-side proxies. The difference is that they are responsible for receiving requests, unpacking incoming messages, unmarshaling the parameters, and calling the appropriate service. They are used in addition for marshaling results and exceptions before sending them to the client.

Class Client-side Proxy	Collaborators • Client • Broker	Class Server-side Proxy	Collaborators • Server • Broker
Responsibility • Encapsulates system-specific functionality. • Mediates between the client and the broker.		Responsibility • Calls services within the server. • Encapsulates system-specific functionality. • Mediates between the server and the broker.	

When results or exceptions are returned from a server, the Client-side proxy receives the incoming message from the broker, unmarshals the data and forward it to the client.

➤ In our CIS example the WWW browsers and httpd servers such as Netscape provide built-in capabilities for communicating with the gateway of the Internet provider, so we do not need to worry about proxies in this case. □

*Bridges*¹⁴ are optional components used for hiding implementation details when two brokers interoperate. Suppose a Broker system runs on a heterogeneous network. If requests are transmitted over the

14. We call these components *Bridges* following the terminology of the OMG in the CORBA 2 specification.

network, different brokers have to communicate independently of the different network and operating systems in use. A bridge builds a layer that encapsulates all these system-specific details.

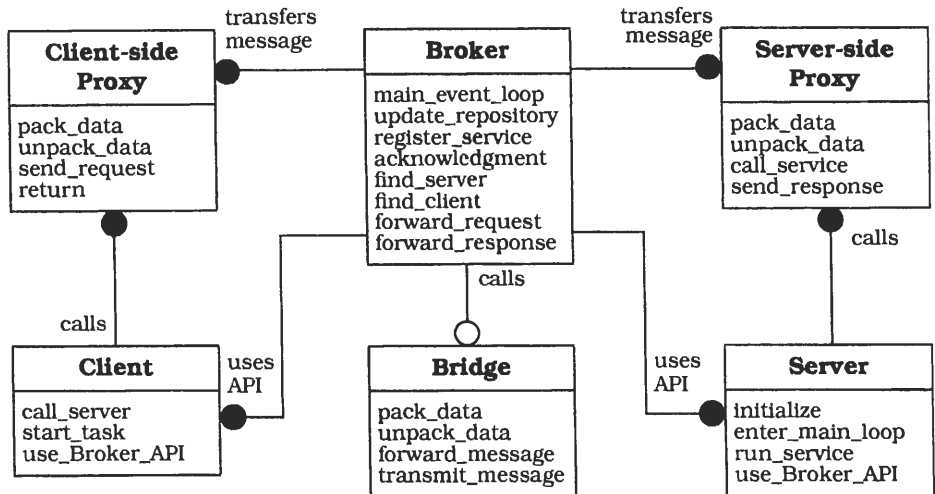
➡ Bridges are not required in our CIS example, because all httpd servers and WWW browsers implement the protocols necessary for remote data exchange such as `http` (hypertext transfer protocol) or `ftp` (file transfer protocol). □

Class Bridge	Collaborators • Broker • Bridge
Responsibility • Encapsulates network-specific functionality. • Mediates between the local broker and the bridge of a remote broker.	

There are two different kinds of Broker systems: those using direct communication and those using indirect communication. To achieve better performance, some broker implementations only establish the initial communication link between a client and a server, while the rest of the communication is done directly between participating components—messages, exceptions and responses are transferred between client-side proxies and server-side proxies without using the broker as an intermediate layer. This direct communication approach requires that servers and clients use and understand the same protocol. In this pattern description we focus on the Indirect Broker variant, where all messages are passed through the broker. The Client-Dispatcher-Server pattern (323) describes the important aspects of the direct variant of the Broker pattern.

➡ Our CIS example implements the indirect communication variant, because browsers and servers can only collaborate using Internet gateways. There is one place in CIS however where we use the direct communication variant instead—Java applets loaded from the network may connect directly to the WWW server from which they came using a socket connection. □

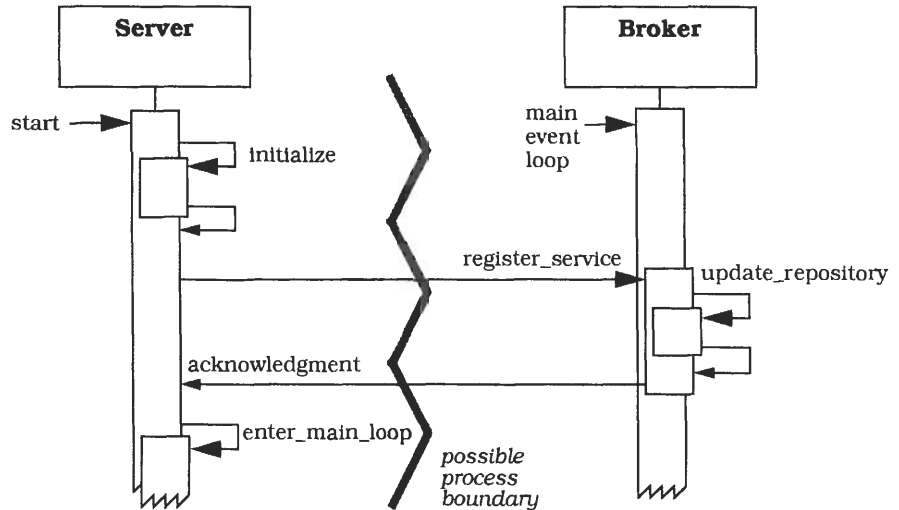
The following diagram shows the objects involved in a Broker system:



Dynamics This section focuses on the most relevant scenarios in the operation of a Broker system.

Scenario I illustrates the behavior when a server registers itself with the local broker component:

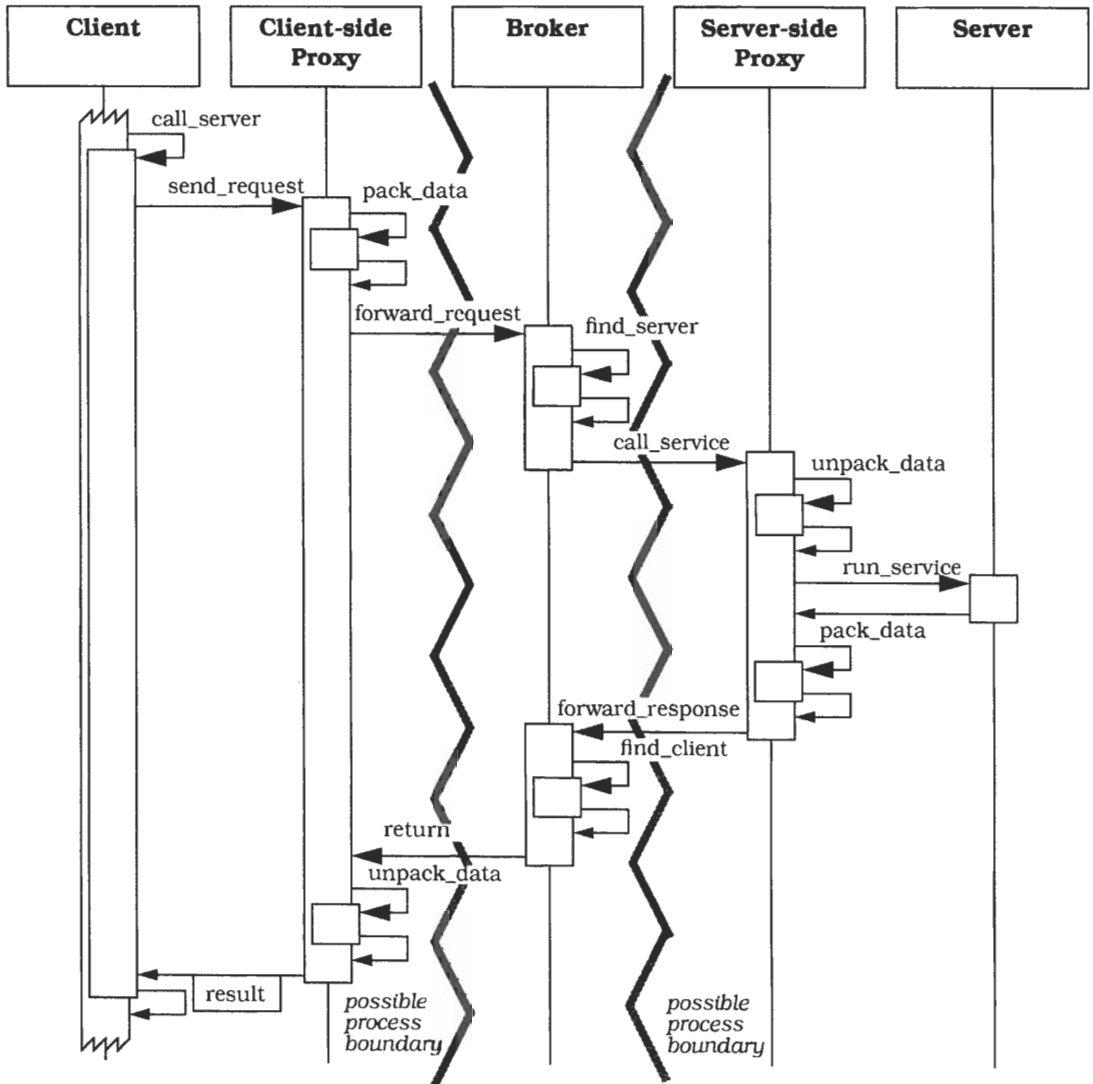
- The broker is started in the initialization phase of the system. The broker enters its event loop and waits for incoming messages.
- The user, or some other entity, starts a server application. First, the server executes its initialization code. After initialization is complete, the server registers itself with the broker.
- The broker receives the incoming registration request from the server. It extracts all necessary information from the message and stores it into one or more repositories. These repositories are used to locate and activate servers. An acknowledgment is sent back.
- After receiving the acknowledgment from the broker, the server enters its main loop waiting for incoming client requests.



Scenario II illustrates the behavior when a client sends a request to a local server. In this scenario we describe a synchronous invocation, in which the client blocks until it gets a response from the server. The broker may also support asynchronous invocations, allowing clients to execute further tasks without having to wait for a response.

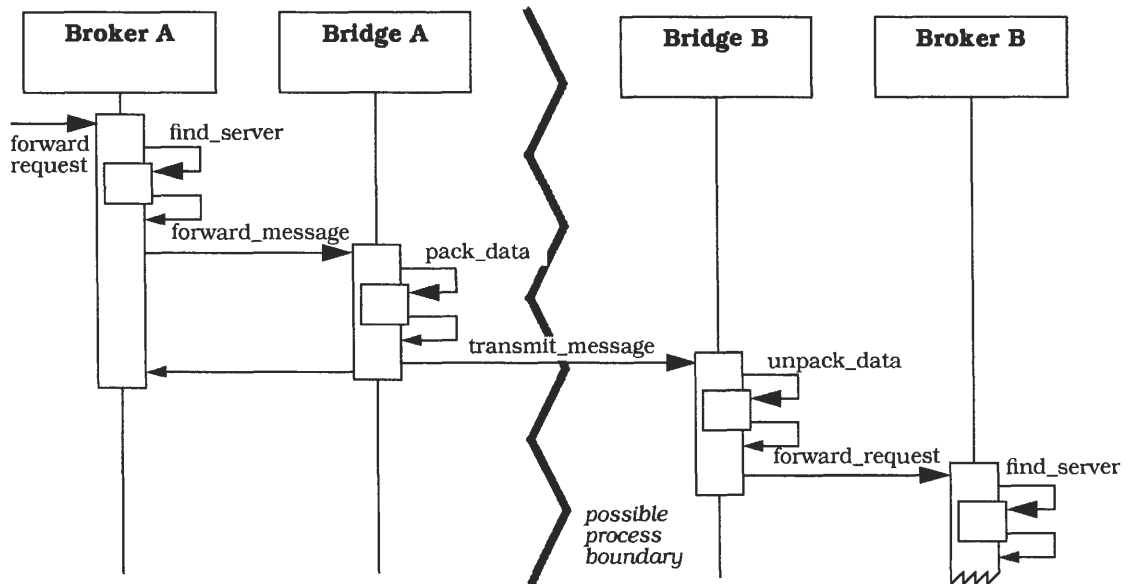
- The client application is started. During program execution the client invokes a method of a remote server object.
- The client-side proxy packages all parameters and other relevant information into a message and forwards this message to the local broker.
- The broker looks up the location of the required server in its repositories. Since the server is available locally, the broker forwards the message to the corresponding server-side proxy. For the remote case, see the following scenario.
- The server-side proxy unpacks all parameters and other information, such as the method it is expected to call. The server-side proxy invokes the appropriate service.
- After the service execution is complete, the server returns the result to the server-side proxy, which packages it into a message with other relevant information and passes it to the broker.

- The broker forwards the response to the client-side proxy.
- The client-side proxy receives the response, unpacks the result and returns to the client application. The client process continues with its computation.



Scenario III illustrates the interaction of different brokers via bridge components:

- Broker A receives an incoming request. It locates the server responsible for executing the specified service by looking it up in the repositories. Since the corresponding server is available at another network node, the broker forwards the request to a remote broker.
- The message is passed from Broker A to Bridge A. This component is responsible for converting the message from the protocol defined by Broker A to a network-specific but common protocol understood by the two participating bridges. After message conversion, Bridge A transmits the message to Bridge B.
- Bridge B maps the incoming request from the network-specific format to a Broker B-specific format.
- Broker B performs all the actions necessary when a request arrives, as described in the first step of this scenario.



Implementation To implement this pattern, carry out the following steps:

- 1 *Define an object model, or use an existing model.* Your choice of object model has a major impact on all other parts of the system under development. Each object model must specify entities such as object names, objects, requests, values, exceptions, supported types, type extensions, interfaces and operations. In this first step you should only consider semantic issues. If the object model has to be extensible, prepare the system for future enhancements. For example, specify a basic object model and how it can be refined systematically using extensions. More information on this topic is available in [OMG92].

The description of the underlying computational model is a key issue in designing an object model. You need to describe definitions of the state of server objects, definitions of methods, how methods are selected for execution and how server objects are generated and destroyed. The state of server objects and their method implementations should not be directly accessible to clients. Clients may only change or read the server's state indirectly by passing requests to the local broker. With this separation of interfaces and server implementations the so-called 'remoting' of interfaces becomes possible—clients use the client-side proxies as server interfaces that are completely decoupled from the server implementations, and thus from the concrete implementations of the server interfaces.

- 2 *Decide which kind of component-interoperability the system should offer.* You can design for interoperability either by specifying a binary standard or by introducing a high-level interface definition language (IDL). An IDL file contains a textual description of the interfaces a server offers to its clients. The binary approach needs support from your programming language. For example, binary method tables are available in Microsoft Object Linking and Embedding (OLE) [Bro94]. These tables consist of pointers to method implementations, and enable clients to call methods indirectly using pointers. Access to OLE objects is only supported by compilers or interpreters that know the physical structure of these tables.

In contrast to the binary approach, the IDL approach is more flexible in that an IDL mapping may be implemented for any programming language. Sometimes both approaches are used in combination, as in IBM's System Object Model (SOM) [Cam94].

An IDL compiler uses an IDL file as input and generates programming-language code or binary code. One part of this generated code is required by the server for communicating with its local broker, another part is used by the client for communicating with its local broker. The broker may use the IDL specification to maintain type information about existing server implementations.

Whenever interoperability is provided as a binary standard, every semantic concept of the object model must be associated with a binary representation. However, if you supply an interface definition language for interoperability, you can map the semantic concepts to programming language representations. For example, object handles may be represented by C++ pointers and data types may be mapped to appropriate C++ types.

One question remains—when should a Broker system expose interfaces with an interface definition language, and when by a binary standard? The rationale for the first approach is to gain more flexibility for the broker's implementation—every implementation of the Broker architecture may define its own protocol for the interaction between the broker and other components. It is the task of the IDL to provide a mapping to the local broker protocol. When following a binary approach, you need to define binary representations such as method tables for invoking remote services. This often leads to greater efficiency, but requires all brokers to implement the same kind of protocol when communicating with clients and servers.

- 3 *Specify the APIs the broker component provides for collaborating with clients and servers.* On the client side, functionality must be available for constructing requests, passing them to the broker and receiving responses. Decide whether clients should only be able to invoke server operations statically, allowing clients to bind the invocations at compile-time. If you want to allow dynamic invocations¹⁵ of servers as well, this has a direct impact on the size or number of APIs. For example, you need some way of asking the broker about existing server objects. You can implement this with the help of a meta-level schema, as described in the Reflection pattern (193).

15. Dynamic invocations are method calls that are dynamically constructed at runtime using API functions as well as type information. In contrast, static invocations are hard-coded into the source code.

You have to offer operations to clients, so that they are capable of constructing requests at run-time. The server implementations use API functions primarily for registering with the broker. Brokers use repositories to maintain the information. These repositories may be available as external files, so that servers can register themselves before system start-up. Another approach is to implement the repository as an internal part of the broker component. Here, the broker must offer an API that allows servers to register at run-time. Since the broker needs to identify these servers when requests arrive, an appropriate identification mechanism is necessary. In other words, the broker component is responsible for associating server object identifiers with server object implementations. The server-side API of the broker must therefore be able to generate system-unique identifiers.

If clients, servers and the broker are running as distinct processes, the API functions need to be based on an efficient mechanism for inter-process communication between clients, servers and the local broker.

- 4 *Use proxy objects to hide implementation details from clients and servers.* On the client side, a local proxy (263) object represents the remote server object called by the client. On the server side, a proxy is used for playing the role of the client. Proxy objects have the following responsibilities:
 - Client-side proxies package procedure calls into messages and forward these messages to the local broker component. In addition, they receive responses and exceptions from the local broker and pass them to the calling client. You must specify an internal message protocol for communication between proxy and broker to support this.
 - Server-side proxies receive requests from the local broker and call the methods in the interface implementation of the corresponding server. They forward server responses and exceptions to the local broker after packaging them, according to an internal message protocol.

Note that proxies are always part of the corresponding client or server process.

Proxies hide implementation details by using their own inter-process communication mechanism to communicate with the broker component. They may also implement the marshaling and unmarshaling of parameters and results into/from a system-independent format.

If you follow the IDL approach for interoperability, proxy objects are automatically available, because they can be generated by an IDL compiler. If you use a binary approach, the creation and deletion of proxy objects can happen dynamically.

- 5 *Design the broker component* in parallel with steps 3 and 4. In this step we describe how to develop a broker component that acts as a messenger for every message passed from a client to a server and vice-versa. To increase the performance of the whole system, some implementations do not transmit messages via the broker. In these systems most of the work is done by the proxies, while the broker is still responsible for establishing the initial communication link between clients and servers. A direct communication between client and server is only possible when both of them can use the same protocol. We call such systems *Direct Communication Broker systems* (see Variants section).

During design and implementation, iterate systematically through the following steps:

- 5.1 Specify a detailed *on-the-wire* protocol for interacting with client-side proxies and server-side proxies. Plan the mapping of requests, responses, and exceptions to your internal message protocol. In an on-the-wire protocol, the internal message protocol handles the mapping of higher-level structures such as parameter values, method names and return values to corresponding structures specified by the underlying inter-process communication mechanism.
- 5.2 A local broker must be available for every participating machine in the network. If requests, responses or exceptions are transferred from one network node to another, the corresponding local brokers must communicate with each other using an on-the-wire protocol. Use bridges to hide details such as network protocols and operating system specifics from the broker. The broker must also maintain a repository to locate the remote brokers or gateways to which it

forwards messages. You may encode the routing information for finding remote brokers as a part of the server or client identifier. Broadcast communication is another (potentially inefficient) way to locate the network node where a server or client resides.

- 5.3 When a client invokes a method of a server, the Broker system is responsible for returning all results and exceptions back to the original client. In other words, the system must remember which client has sent the request. In the *Direct Communication variant* (see the Variants section) there is no need to remember the originator of an invocation, because the client and the server are directly connected through a communication channel. In Indirect Broker systems you can choose between different means of remembering the sender of a request. For example, you may specify the client's address as an additional, invisible parameter of the request or message.
- 5.4 If the proxies (see step 4) do not provide mechanisms for marshaling and unmarshaling parameters and results, you must include that functionality in the broker component.
- 5.5 If your system supports asynchronous communication between clients and servers, you need to provide *message buffers* within the broker or within the proxies for the temporary storage of messages.
- 5.6 Include a *directory service* for associating local server identifiers with the physical location of the corresponding servers in the broker. For example, if the underlying inter-process communication protocol is based on TCP/IP, you could use an Internet port number as the physical server location.
- 5.7 When your architecture requires system-unique identifiers to be generated dynamically during server registration, the broker must offer a *name service* for instantiating such names.
- 5.8 If your system supports *dynamic method invocation* (see step 3), the broker needs some means for maintaining type information about existing servers. A client may access this information using the broker APIs to construct a request dynamically. You can implement such type information by instantiating the Reflection pattern (193). In this, metaobjects maintain type information that is accessible by a metaobject protocol.

5.9 Consider the case in which something fails. In a distributed system two levels of errors may occur:

- A component such as a server may run into an error condition. This is the same kind of error you encounter when executing conventional non-distributed applications.
- The communication between two independent processes may fail. Here the situation is more complicated, since the communicating components are running asynchronously.

Plan the broker's actions when the communication with clients, other brokers or servers fails. For example, some brokers resend a request or response several times until they succeed. If you use an *at-most-once semantic*¹⁶, you have to make sure that a request is only executed once even if it is resent. Do not forget the case in which a client tries to access a server that either does not exist, or which the client is not allowed to access. Error handling is an important topic when implementing a distributed system. If you forget to handle errors in a systematic way, testing and debugging of client applications and servers becomes an extremely tedious job.

- 6 *Develop IDL compilers.* Whenever you implement interoperability by providing an interface definition language, you need to build an *IDL compiler* for every programming language you support. An IDL compiler translates the server interface definitions to programming language code. When many programming languages are in use, it is best to develop the compiler as a *framework* that allows the developer to add his own code generators.

Example Resolved Our example CIS system offers different kinds of services. For example, a separate server workstation provides all the information related to public transport. Another server is responsible for collecting and publishing information on vacant hotel rooms. A tourist may be interested in retrieving information from several

16. When supporting at-most-once semantics your system has to guarantee that any request either fails, or is executed only once. If you implement other semantics instead such as *at-least-once*, the same request may be resent and executed several times. This strategy is only applicable to *idempotent* services, where overall consistency is not damaged by executing a service more than once. A typical example of an idempotent service is a function that assigns an initial value to a variable.

hotels, so we decide to provide this data on a single workstation. Every hotel can connect to the workstation and perform updates.

A tourist is capable of booking hotel rooms on-line from anywhere in the Internet using CGI scripts. Payments for hotel reservations are charged on-line by credit card. For security reasons we include encryption mechanisms for such transactions. Additional httpd servers are available to provide extra services such as flight booking or train reservations, the ordering of tickets or the retrieval of information about museums and other places of interest.

Each CIS terminal executes a WWW browser. This allows us to use inexpensive PCs and Internet PCs as terminals. The httpd servers run on fast UNIX and Windows NT workstations to guarantee short response times.

Variants *Direct Communication Broker System.* You may sometimes choose to relax the restriction that clients can only forward requests through the local broker for efficiency reasons. In this variant clients can communicate with servers directly. The broker tells the clients which communication channel the server provides. The client can then establish a direct link to the requested server. In such systems, the proxies take over the broker's responsibility for handling most of the communication activities. A similar argument applies to *off-board* communication: here clients address the remote broker directly, using bridges when appropriate, as opposed to sending requests to their local broker for forwarding to the remote server's broker.

Message Passing Broker System. This variant is suitable for systems that focus on the transmission of data, instead of implementing a Remote Procedure Call abstraction¹⁷. Using this variant, servers use the type of a message to determine what they must do, rather than offering services that clients can invoke. In this context, a message is a sequence of raw data together with additional information that specifies the type of a message, its structure and other relevant attributes.

Trader System. A client request is usually forwarded to exactly one uniquely-identified server. In some circumstances, services and not

17. Brokers offering RPC (Remote Procedure Call) interfaces are typically built using message-passing interfaces.

servers are the targets to which clients send their requests. In a Trader system, the broker must know which server(s) can provide the service, and forward the request to an appropriate server. Client-side proxies therefore use *service identifiers* instead of server identifiers to access server functionality. The same request might be forwarded to more than one server implementing the same service.

Adapter Broker System. You can hide the interface of the broker component to the servers using an additional layer, to enhance flexibility. This *adapter* layer is a part of the broker and is responsible for registering servers and interacting with servers. By supplying more than one adapter, you can support different strategies for server granularity and server location. For example, if all the server objects accessed by an application are located on the same machine and are implemented as library objects, a special adapter could be used to link the objects directly to the application. Another example is the use of an object-oriented database for maintaining objects. Since the database is responsible for providing methods and storing objects, there may be no need to register objects explicitly. In such a scenario, you could provide a special database adapter. See also [OMG92].

Callback Broker System. Instead of implementing an active communication model in which clients produce requests and servers consume them, you can also use a *reactive* model. The reactive model is event-driven, and makes no distinction between clients and servers. Whenever an event arrives, the broker invokes the callback method of the component that is registered to react to the event. The execution of the method may generate new events that in turn cause the broker to trigger new callback method invocations. For more details on this variant, see [Sch94].

There are several ways of combining the above variants. For example, you can implement a Direct Communication Broker system and combine it with the Trader variant. In such a system an incoming client request causes the broker to select one server among those that provide the requested service. The broker then establishes a direct link between the client and the selected server.

Known Uses **CORBA.** The Broker architectural pattern was used to specify the Common Object Request Broker Architecture (CORBA) defined by the Object Management Group. CORBA is an object-oriented technology

for distributing objects on heterogeneous systems. An interface definition language is available to support the interoperability of client and server objects [OMG92]. Many CORBA implementations realize the *Direct Communication Broker System* variant, for example IONA Technologies' Orbix [Iona95].

IBM SOM/DSOM. [Cam94] represents a CORBA-compliant Broker system. In contrast to many other CORBA implementations, it implements interoperability by combining the CORBA interface definition language with a binary protocol. SOM's binary approach supports subclassing from existing binary parent classes. You can implement a class in SOM in one programming language and derive a subclass from it in another language.

Microsoft's **OLE 2.x** technology provides another example of the use of the Broker architectural pattern. While CORBA guarantees interoperability using an interface definition language, OLE 2.x defines a binary standard for exposing and accessing server interfaces [Bro94].

The **World Wide Web** is the largest available Broker system in the world. Hypertext browsers such as HotJava, Mosaic, and Netscape act as brokers and WWW servers play the role of service providers.

ATM-P. We implemented the *Message Passing Broker System* variant [ATM93] in a Siemens in-house project to build a telecommunication switching system based on ATM (Asynchronous Transfer Mode).

Consequences The Broker architectural pattern has some important **benefits**:

Location Transparency. As the broker is responsible for locating a server by using a unique identifier, clients do not need to know where servers are located. Similarly, servers do not care about the location of calling clients, as they receive all requests from the local broker component.

Changeability and extensibility of components. If servers change but their interfaces remain the same, it has no functional impact on clients. Modifying the internal implementation of the broker, but not the APIs it provides, has no effect on clients and servers other than performance changes. Changes in the communication mechanisms used for the interaction between servers and the broker, between clients and the broker, and between brokers may require you to recompile clients, servers or brokers. However, you will not need to

change their source code. Using proxies and bridges is an important reason for the ease with which changes can be implemented.

Portability of a Broker system. The Broker system hides operating system and network system details from clients and servers by using indirection layers such as APIs, proxies and bridges. When porting is required, it is therefore sufficient in most cases to port the broker component and its APIs to a new platform and to recompile clients and servers. Structuring the broker component into layers is recommended, for example according to the Layers architectural pattern (31). If the lower-most layers hide system-specific details from the rest of the broker, you only need to port these lower-most layers, instead of completely porting the broker component.

Interoperability between different Broker systems. Different Broker systems may interoperate if they understand a common protocol for the exchange of messages. This protocol is implemented and handled by bridges, which are responsible for translating the broker-specific protocol into the common protocol, and vice versa.

Reusability. When building new client applications, you can often base the functionality of your application on existing services. Suppose you are going to develop a new business application. If components that offer services such as text editing, visualization, printing, database access or spreadsheets are already available, you do not need to implement these services yourself. It may instead be sufficient to integrate these services into your applications.

The Broker architectural pattern imposes some **liabilities**:

Restricted efficiency. Applications using a Broker implementation are usually slower than applications whose component distribution is static and known. Systems that depend directly on a concrete mechanism for inter-process communication also give better performance than a Broker architecture, because Broker introduces indirection layers to enable it to be portable, flexible and changeable.

Lower fault tolerance. Compared with a non-distributed software system, a Broker system may offer lower fault tolerance. Suppose that a server or a broker fails during program execution. All the applications that depend on the server or broker are unable to continue successfully. You can increase reliability through replication of components.

The following aspect gives **benefits** as well as **liabilities**:

Testing and Debugging. A client application developed from tested services is more robust and easier itself to test. However, debugging and testing a Broker system is a tedious job because of the many components involved. For example, the cooperation between a client and a server can fail for two possible reasons—either the server has entered an error state, or there is a problem somewhere on the communication path between client and server.

See also The *Forwarder-Receiver* pattern (307) encapsulates inter-process communication between two components. On the client side a *forwarder* receives a request and addressee from the client and handles the mapping to the IPC (inter-process communication) facility used. The receiver on the server side unpacks and delivers the message to the server. There is no broker component in this pattern. It is simpler to implement and results in smaller implementations than the Broker pattern, but is also less flexible.

The *Proxy* pattern (263) comes in several flavors, the *remote* case being one of them. A remote proxy is often used in conjunction with a forwarder. The proxy encapsulates the interface and remote address of the server. The forwarder takes the message and transforms it into IPC-level code.

The *Client-Dispatcher-Server* pattern (323) is a lightweight version of the Direct Communication Broker variant. A *dispatcher* allocates, opens and maintains a direct channel between client and server.

The *Mediator* design pattern [GHJV95] replaces a web of inter-object connections by a star configuration in which the central *mediator* component encapsulates collective behavior by defining a common interface for communicating with objects. As with the Broker pattern, the Mediator pattern uses a hub of communication, but it also has several major differences. The Broker pattern is a large-scale infrastructure paradigm—it is not used for building single applications, but rather serves as a platform for whole families of applications. It is not restricted to processing local computation, and dispatches and monitors requests without regard to the sender or the content of the

request. In contrast, the Mediator pattern encapsulates application semantics by checking what a request is about and possibly where it came from—only then does it decide what to do. It may return a message to the sender, fulfill the request on its own, or involve more than one other component.

Credits We wish to thank the participants of the workshop on patterns for concurrent and distributed systems at OOPSLA '95 for reviewing the Broker pattern. Special credit is due to Jim Coplien, David DeLano, Doug Schmidt and Steve Vinoski, who reviewed early version of the Broker description and contributed several fruitful suggestions and hints.